Coinbase SEA: Insights to Impact Challenge

Armaan Singla · ~1,380 words (headings, tables and AI disclosure included; code excluded)

Note: fictional scenarios, synthetic data; SQL written for Snowflake but run only through a SQLite port. Code and tests: github.com/armaansingla14/coinbase-insights-to-impact.

Exercise 1: Design the Data Model

What the PM needs: one row per user, so Growth can see which step loses the most people, compare sign-up cohorts fairly, and tell real drop-off from missing instrumentation.

Layer	Grain	Job
`stg_app`, `stg_kyc`, `stg_ledger`	source event	UTC timestamps, normalized IDs, 5 canonical steps
`int_identity_map`	source ref	One account per ref, else `ambiguous_link` / `orphan_link`
`int_onboarding_quarantine`	rejected event	Rejected events, with a reason
`fct_onboarding_events`	user × step	First occurrence; retries counted
`fct_onboarding_funnel`	user	Observed timestamps plus a state per step

Keys. The account `user_uuid` is the only join key; emails and devices only map *to* it through deterministic links. UUIDs compare case-insensitively; device IDs match exactly. A ref pointing to two accounts is quarantined, not guessed. At Scotiabank, connecting NPS feedback with login telemetry across 337 outage cases taught me to confirm two records share a user and time window before trusting a join, and to study what didn't match.

Missing and out-of-order events. I never invent timestamps. Each step gets a state:

- `observed`: recorded by the source.
- `inferred`: implied by a later mandatory step (assumed: email and KYC precede funding; product to confirm).
- `rejected_event`: an event existed but failed parsing (instrumentation, not behavior).
- `not_reached`: no evidence.

I model deposits and trades as separate funding routes (product to confirm), so funding = deposit *or* trade. Out-of-order events keep real timestamps and are flagged; late events rebuild affected users idempotently. A DST local time that doesn't exist is quarantined; a repeated hour takes the earlier instant, flagged.

Step states show where users stop; for 7-day funding, the PM runs:

```
-- ex1/pm_funnel_query.sql (complete query; header comments omitted)
WITH users AS (
    SELECT
        DATEADD('day', 1 - DAYOFWEEKISO(signup_at), DATE(signup_at))  AS cohort_week,
        COALESCE(first_deposit_after_signup_at  -- >= effective signup anchor by construction
                 < DATEADD('day', 7, signup_at), FALSE)               AS deposit_in_7d,
        COALESCE(first_trade_after_signup_at
                 < DATEADD('day', 7, signup_at), FALSE)               AS trade_in_7d
    FROM analytics.fct_onboarding_funnel
)
SELECT cohort_week,
       COUNT(*)                                                AS signups,
       -- COUNT_IF returns NULL when no row matches, so default each count to 0
       COALESCE(COUNT_IF(deposit_in_7d), 0)                    AS observed_deposit_7d,
       COALESCE(COUNT_IF(deposit_in_7d OR trade_in_7d), 0)  AS funding_7d,
       COALESCE(COUNT_IF(trade_in_7d), 0)                      AS trade_7d
FROM users
WHERE DATEADD('day', 14, cohort_week::TIMESTAMP_NTZ) <= SYSDATE()      -- mature weeks only
GROUP BY cohort_week
ORDER BY cohort_week;
```

Window: `[signup, signup + 7 days)`, on the earliest deposit or trade *at or after* sign-up, so a pre-sign-up trade can't hide a valid one; original timestamps stay for audit. Weeks appear once all their users have had 7 days; the Monday week ignores `WEEK_START`.

Checks: unique grain; ≥ 99% identity match (staged = used + quarantined); one UUID per `account_id`; sign-ups within 0.5% of accounts daily; timestamp sanity; freshness under 6h; out-of-order and assumed-UTC alerts.

Tested with 58 SQLite tests, including hand-calculated edge cases (day-2 trade with day-8 deposit, trades before sign-up, the exact day-7 boundary) and metamorphic tests.

Exercise 2: The Data Doesn't Add Up

Why the numbers differ is a definitions question before it's a data question. I'd reconcile at one grain, distinct accounts, never rows. My two hours:

1. **30 min:** rows vs. distinct IDs per system, starting with C's duplicates, plus time ranges.
2. **45 min:** ask each owner what "signup" means (created or verified, UTC or Pacific day, test/bot filters).
3. **30 min:** bridge the systems and count whatever stays unexplained.
4. **15 min:** message the PM.

After Friday: dedupe at ingestion and name one official metric.

Synthetic extracts with planted causes test the method, not what's true at Coinbase:

	What I know
Given	Same day: A 52K, B 45K, C 57K
Assumed	Account ID is shared across systems; "signup" means account created
Synthetic run	Every planted cause recovered, 0 unexplained: C's 5,000 replayed rows, B's Pacific day and verified-only filter, A's 2,000 test accounts
Still a hypothesis	That the real gaps have these causes

```
# Excerpt: why_not_in_b() in ex2/reconcile_signups.py
    checks = [(OUTSIDE, not in_window(ts, utc_day)),
              ("test/internal account", is_test(r)),
              (f"created before {pt_day[0]:%H:%M} UTC (previous Pacific day)", ts < pt_day[0]),
              ("created after the Pacific day ended", ts >= pt_day[1]),
              ("email not verified", not (r.get("email_verified_at") or "").strip())]
    return [why for why, hit in checks if hit] or ["UNEXPLAINED"]
```

Output on the synthetic data (abridged):

```
C: 4,000 user_ids duplicated; 5,000 extra rows
   columns that differ within a group: {'ingest_batch_id': 4000}
   extra rows in *_retry batches: 5,000 of 5,000 across 12 batches
B:  45,000 rows |  45,000 distinct | 09-24 07:00 -> 09-25 06:59 UTC
A raw                                                  52,000
  - created before 07:00 UTC (previous Pacific day)   -12,000
  - email not verified                                 -5,500
  - test/internal account                              -2,000
  + created on next UTC day (still Pacific report day)  +12,500
= B distinct accounts                                  45,000
  created before 07:00 UTC (previous Pacific day) & email not verified: 1,207
  test/internal account & created before 07:00 UTC (previous Pacific day): 608
UNEXPLAINED users: 0
```

Each user gets one reason in a fixed order; 1,815 matched several, so the buckets aren't independent causes.

What I'd send the PM:

> "Hey, I looked into why the three signup numbers don't match. I know you need one number for Friday. They seem to count different things: C looks like it has duplicate rows, and B a Pacific-time day with only verified users. I'd define signups as **accounts created on the UTC day, excluding internal test accounts**. I'm confirming the causes with each system's owner by Thursday noon. If confirmed, it goes in the deck with the definition footnoted. If not, we use A's 52K labeled 'accounts created, filters unconfirmed, under review.'"

What I flag: possible replays in C, B's name hiding a narrower definition, test accounts in A.

Exercise 3: The Inherited Pipeline

What matters: the PM needs a country view by Friday, and other dashboards rely on this metric. The snippet already groups by country, so the view is probably lost downstream (rollup, semantic layer or dashboard); I'd find that first.

#	Issue	Why it matters
1	`devices` join fans out	Each trade repeats per active device, inflating `completed_trades`
2	`d.is_active = 1`	Drops traders with no active device; current state, so reruns change past weeks
3	`status = 1 OR status = 3`	Undocumented codes; `active_traders` counts *any* status, even failed
4	`users` history rows	A user who moved appears in several countries
5	Comma joins	A dropped predicate silently becomes a cross join
6	Hardcoded dates, unknown timezone	Not really weekly; boundaries unclear

At AMD, working with existing security-notice workflows taught me to trace inputs, outputs and rules before changing anything, so I'd run the current query and pin its output first.

How I'd deliver without breaking the metric:

1. Snapshot recent weeks; find every dashboard reading this table.
2. Write characterization tests that pin the current output. Passing means *unchanged*, not *correct*.
3. **Friday:** country on unchanged legacy logic, with a reconciliation line: trade counts add up across countries, but distinct traders don't when a user has several country rows (fixture: 9 vs. 7 global).
4. v2 side by side: one row per user-week, statuses from a lookup, country as of the week's first completed trade (my assumption; the owner may prefer sign-up or current country).
5. Compare, attribute each difference, and switch only after the metric owner signs off.

```
-- Excerpt: one CTE from ex3/v2_fct_user_week_trades.sql
), country_as_of_first_completed AS (
    SELECT w.user_id, w.week_start, u.country,
           ROW_NUMBER() OVER (PARTITION BY w.user_id, w.week_start
                              ORDER BY u.valid_from DESC NULLS LAST, u.country NULLS LAST) AS rn
    FROM user_week w
    JOIN users u
      ON u.user_id = w.user_id
     AND (u.valid_from IS NULL OR u.valid_from <= w.first_completed_at)
    WHERE w.is_active_trader = 1
)
```

To the PM: "You'll have the country cut by Thursday on the current definition. Trader counts can overlap slightly for users who moved, so there's a reconciliation line. Trade counts are inflated for multi-device users until v2 lands."

Using AI: to explain the 400 lines, map dependencies and draft tests; verified by diffing old vs. new at every grain, hand-tracing edge cases and confirming status codes with owners.

Exercise 4: Your Number Is "Wrong"

First, what are we counting? The brief says the legacy 41% "double-counts users who trade on multiple devices," so its numerator counts user-devices while my 34% counts each trader once. That differs from Exercise 3's join, which repeats trades. Before the review I'd still check nothing else differs:

Hypothesis	Test	Supported if
Device-level numerator (given)	Rows per user	Deduplicating gives 34%
Attempted vs. completed trades	Recompute with each status set	Gap closes on completed only
Denominator eligibility (KYC, test, bots)	Diff the two WAU lists	Denominators differ
Week window or timezone	Align boundaries	Gap closes
Segment filters	Diff filter logic	Populations differ
Metric version or freshness	Changelog, load dates	Gap starts at a release

What I'd say to the PM, privately:

> "Can we talk before the review? I get why a discrepancy in the deck is stressful, and I'm not trying to make your job harder. The dashboard counts some traders once per device; mine counts each person once. My worry is leadership making a call on a number we think might be off, because once it's in a report people build on it. Making my table match wouldn't make either number right. I'd rather we explain the difference now than explain later why we used it."

For Friday: headline 34% with 41% footnoted as "legacy, double-counts multi-device traders." If the checks find another difference, the footnote names it too. The PM presents; I don't correct them in the room. If pressed to present a number I know is wrong, I'd raise it with my manager.

After the review: one owned definition in the semantic layer (numerator, denominator, grain, filters, timezone), two cycles in parallel, a review of past decisions near thresholds, restated history with a changelog, and a grain test.

AI Disclosure

I used Claude Code to draft these responses, build the SQL and Python, generate synthetic data and write tests, and ChatGPT to review the repository, find edge cases and help draft my correction requests and personal answers. I directed the project, requested corrections and reviewed the results. All numbers come from synthetic data; explanations for real systems are hypotheses.